

Project: MediServe360 - Hospital Administration & Patient Care Platform

1. Introduction

MediServe360 is a web-based Hospital Administration and Patient Care Platform designed for hospitals, clinics, and healthcare networks to unify patient registration, appointment scheduling, billing, and compliance reporting. The platform consolidates patient demographics, medical records, appointments, and financial transactions to provide end-to-end visibility, streamlined workflows, and adherence to healthcare regulations.

It supports RESTful APIs with Angular/React frontends, and backend implementations in Java Spring Boot or ASP.NET Core. The architecture emphasizes reliability, secure handling of patient data, and compliance with HIPAA/GDPR standards.

Actors / Users

- **Doctor:** Views patient records, updates medical notes, and manages appointments.
- **Nurse:** Tracks patient vitals, updates care notes, and manages ward assignments.
- **Receptionist:** Registers patients, schedules appointments, and manages queues.
- **Billing Officer:** Handles invoices, payments, and insurance claims.
- **Compliance Officer:** Reviews audit logs and generates regulatory reports.
- **Admin:** Configures departments, users, and system settings.

2. Module Overview

2.1 Identity & Access Management

Authentication, RBAC, audit trails.

2.2 Patient Registration & Records

Patient demographics, medical history, and record updates.

2.3 Appointment & Scheduling

Doctor availability, patient appointments, and scheduling.

2.4 Billing & Insurance Management

Invoices, payments, and insurance claim tracking.

2.5 Ward & Bed Management

Ward allocation, bed occupancy, and discharge workflows.

2.6 Compliance & Audit Management

Audit logs, regulatory reporting, and compliance dashboards.

2.7 Analytics & Reporting

Operational dashboards, KPIs, and compliance reports.

2.8 Notifications & Alerts

Real-time alerts for appointments, billing, and compliance deadlines.

3. Architecture Overview

- **Frontend:** Angular or React SPA for dashboards and workflows.
- **Backend:** RESTful services (Spring Boot / ASP.NET Core).
- **Database:** Relational DB (SQL Server/MySQL/PostgreSQL).
- **Integration:** ERP, insurance systems, and regulatory portals.
- **Security:** TLS, encryption at rest, RBAC, audit logging.

4. Module Wise Design

4.1 Identity & Access Management Module

Features

- User provisioning and role assignment
- Secure login with RBAC
- Session management and password policies
- Immutable audit logs

Entities

- **User**(UserID, Name, Role, Email, Phone)
- **AuditLog**(AuditID, UserID, Action, Timestamp)

4.2 Patient Registration & Records Module

Features

- Patient registration and demographic capture
- Medical history updates and record storage
- Search and retrieval of patient records
- Compliance with healthcare data standards

Entities

- **Patient**(PatientID, Name, DOB, Gender, ContactInfo, MedicalHistory, Status)

4.3 Appointment & Scheduling Module

Features

- Doctor availability management
- Patient appointment booking and rescheduling
- Queue management and reminders
- Calendar integration

Entities

- **Appointment**(AppointmentID, PatientID, DoctorID, Date, Time, Status)
- **Doctor**(DoctorID, Name, Department, AvailabilitySchedule)

4.4 Billing & Insurance Management Module

Features

- Invoice generation and payment tracking
- Insurance claim submission and status tracking
- Refunds and adjustments
- Financial reporting

Entities

- **Invoice**(InvoiceID, PatientID, Amount, Date, Status)
- **InsuranceClaim**(ClaimID, PatientID, PolicyNumber, Amount, Status)

4.5 Ward & Bed Management Module

Features

- Ward allocation and bed assignment
- Occupancy tracking
- Discharge workflows
- Ward capacity reporting

Entities

- **Ward**(WardID, Name, Capacity, Status)
- **Bed**(BedID, WardID, PatientID, Status)

4.6 Compliance & Audit Management Module

Features

- Audit log review
- Regulatory reporting generation
- Compliance dashboards

Entities

- **ComplianceReport**(ReportID, Scope, Metrics, GeneratedDate)

4.7 Analytics & Reporting Module

Features

- Operational dashboards for patient flow and billing

- KPI tracking (Occupancy Rate, Appointment Fulfillment, Claim Success Rate)
- Compliance reporting
- Trend analysis

Entities

- **KPIReport**(ReportID, Scope, Metrics, GeneratedDate)

4.8 Notifications & Alerts Module

Features

- Real-time alerts for appointments, billing, and compliance deadlines
- Escalation workflows
- Configurable notification channels (in-app, email, SMS)
- Notification history

Entities

- **Notification**(NotificationID, UserID, Message, Category, Status, CreatedDate)

5. Deployment Strategy

- **Local / Dev:** Angular/React UI, Spring Boot/.NET backend, local RDBMS.
- **Staging:** Containerized services with synthetic patient and billing data.
- **Production:** On-premise or hosted deployment with API gateway, managed RDBMS with replicas.
- **High Availability:** Multi-server deployments, automated backups, disaster recovery playbooks.

6. Database Design

Tables

- **User**(UserID, Name, Role, Email, Phone)
- **AuditLog**(AuditID, UserID, Action, Timestamp)
- **Patient**(PatientID, Name, DOB, Gender, ContactInfo, MedicalHistory, Status)
- **Appointment**(AppointmentID, PatientID, DoctorID, Date, Time, Status)
- **Doctor**(DoctorID, Name, Department, AvailabilitySchedule)
- **Invoice**(InvoiceID, PatientID, Amount, Date, Status)
- **InsuranceClaim**(ClaimID, PatientID, PolicyNumber, Amount, Status)
- **Ward**(WardID, Name, Capacity, Status)

- **Bed**(BedID, WardID, PatientID, Status)
- **ComplianceReport**(ReportID, Scope, Metrics, GeneratedDate)
- **KPIReport**(ReportID, Scope, Metrics, GeneratedDate)
- **Notification**(NotificationID, UserID, Message, Category, Status, CreatedDate)

7. User Interface Design

- **Doctor Dashboard:** Patient records, appointments, medical notes.
- **Nurse Console:** Ward assignments, patient vitals, care notes.
- **Receptionist Dashboard:** Registration, appointment scheduling, queue management.
- **Billing Console:** Invoices, payments, insurance claims.
- **Compliance Dashboard:** Audit logs, regulatory reports.
- **Admin Console:** Department setup, user roles, system configuration.

8. Non-Functional Requirements

- **Performance:** Handle up to **15,000 concurrent users**; appointment booking latency under **300 ms**.
- **Security:** RBAC, TLS, encryption at rest, HIPAA/GDPR compliance, immutable audit logs.
- **Scalability:** Horizontal scaling for patient records and billing services.
- **Availability:** Target **99.9% uptime** for patient and billing services.
- **Maintainability:** Modular APIs, versioning, automated migrations, CI/CD pipelines.
- **Observability:** Centralized logs, metrics, and alerting.
- **Compliance:** Configurable retention policies, audit trails, regulatory reporting formats.

9. Assumptions & Constraints

- **Phase 1 scope:** Patient registration, appointment scheduling, billing, ward management, and dashboards.
- **Excluded in Phase 1:** AI-based diagnosis support, IoT device integrations, advanced telemedicine features.
- **Data availability:** Patient and billing data feeds available for integration.
- **Integration:** ERP, insurance systems, and regulatory portals via APIs.
- **Implementable:** Using Java Spring Boot or ASP.NET Core, Angular/React, and SQL Server/MySQL/PostgreSQL.